CS 150 Final Project: Financial Literacy Simulation

This is an individual project. You cannot work on this project with a partner. The deadline is firm. No late project submissions will be accepted.

Academic integrity

The submitted files must be your work and only your work. You can refer to your class notes, course slides and code examples, matplotlib documentation, and your lab files. You cannot search the internet or ask AI assistants for solutions, hints, or bug fixes. Imagine this to be an open-notes final exam; it just has a take-home format to give you necessary time to complete it. Do not share code, don't share screens, don't type up or paste someone else's (or AI-generated) code in your submission. Please make sure you can discuss and clarify the logic and functionality of your code; never include code that you don't understand.

Overview

In this project you will design and implement an object-oriented simulation of how money grows (or doesn't) over 40 years for two people: one who is **financially literate** (FL) and one who is **not** (NFL). Both start from the same financial position. Your job is to design reasonable rules for how they behave differently and simulate how their **wealth** evolves over time.

You have **substantial freedom** in how you design your classes, attributes, and methods, as long as you obey a **small required interface** so that your work can be graded consistently. You must also **visualize** the results by plotting how wealth changes over time.

Background: What is financial literacy?

Inspired by the Freakonomics podcast Everything You Always Wanted to Know About Money (But Were Afraid to Ask).

If you've spent any time reading up on education or financial news lately, you've probably come across the term *financial literacy*. The goal behind teaching financial literacy is to help people develop a stronger understanding of basic financial concepts—that way, they can handle their money better.

That’s a worthy goal, especially when you consider a few stats about how the typical American handles money: * Nearly four out of every five U.S. workers live paycheck to paycheck. * Over a quarter never save any money from month to month. * Almost 75% are in some form of debt, and most assume they always will be.

With those numbers, it’s no surprise that leaders in business, education and government want to help spread the benefits of greater financial literacy to as many people as possible.

So, what is financial literacy? Financial literacy is the possession of skills that allow people to make smart decisions with their money. And don’t be misled by the word literacy. Although understanding stats and facts about money is great, no one has truly grasped financial literacy until they can regularly make smart decisions with their money that leads to positive financial outcomes. When you have this skill set, you’re able to understand the major financial issues most people face: emergencies, debts, investments, and beyond. Financially literate people know their way around a budget, know how to use sinking funds, and can tell you the difference between a 401(k) and 529” What do you need to know about financial literacy?.

Want to test your financial literacy? Answer these questions and see how well you did here.

```
    1) “Suppose you had $100 in a savings account and the interest rate was 2% per
year. After 5 years, how much do you think you would have in the account if you left
the money to grow?”
        A) More than $102
        B) Exactly $102
        C) Less than $102
        D) Don’t know

    2) “Imagine that the interest rate on your savings account was 1% per year and
inflation was 2% per year. After 1 year, with the money in this account, would you be
able to buy...”
        A) More than today
        B) Exactly the same as today
        C) Less than today
        D) Don’t know

    3) “Do you think the following statement is true or false?
        Buying a single company stock usually provides a safer return than a stock
mutual fund.”
        A) True
        B) False
        C) Don’t know
```

If you didn’t get all three questions right, you’re not alone. This quiz has been used across many countries and demographics, and the majority do not correctly answer all three questions. Notably, “surveys have demonstrated generally lower levels of financial literacy among women, those with lower levels of education, and certain minority groups such as African-Americans and Hispanics. Such barriers to financial literacy are not a matter of

intelligence. Instead, traditional gender roles regarding accumulating and managing wealth, socio-economic factors such as income and access to education, cultural attitudes on consumption and financial services, and language barriers often affect the development of financial literacy within those demographics. If you'd like to test your own financial literacy in a game/simulation where you earn minimum wage, click here. (It's very hard.)

Introduction to the assignment

The goal of the assignment is to simulate **how much money** a financially literate person versus a non–financially literate person has after **40 years**. They start with the same amount of money, the same debt, the same income, and the same house price. The **small differences in the choices they make** (where they save, how fast they pay off debt, when they buy a house) accumulate over time, so that by the end of the simulation there is a clear gap in wealth between the two.

Financial literacy covers a lot of ground. This project simplifies real life so you can focus on a few big levers. In particular, the assignment is built around three kinds of decisions:

1. **Where to save** (e.g., a low-interest bank account vs. a higher-return option like a mutual fund).
2. **How aggressively to pay down debt** (e.g., paying only the minimum vs. paying extra each month to reduce interest).
3. **When to buy a house** (e.g., buying as soon as you can afford a small down payment vs. waiting until you can put down more (e.g., 20%) and avoid extra costs like private-mortgage interest (PMI)).

You are free to model these (and other factors) in your own way, as long as you follow the required interface and produce the required outputs.

FL and NFL both have: * a "savings" account with a corresponding number that represents the amount in savings they have, * a "checking" account with money that they use to pay rent or mortgage, * "debt" from their credit cards, * a "loan" for when they take out a loan for their house. The total wealth (sometimes called "net worth") is usually defined as: **savings + checking − debt − mortgage loan**.

Income and checking account

FL and NFL both earn $61,000 a year, the median of a middle-class person. For simplicity, we'll assume they receive the money at the start of each year. They put 20% into savings, and 30% into checking. They spend 15% on food, 15% on transportation and 10% on miscellaneous, so the amount that ends up going into checking and savings is actually $30,500.

Savings account

The initial balance of the savings account is $5000 for everyone.

NFL puts the money in a simple savings account, which (like many saving accounts) has a 1% annual interest. NFL doesn't know that since inflation is 2% per year, their money will be worth less and less every year. FL knows about inflation, so they put their money in a mutual fund. Mutual funds typically have an annual interest rate of 7%.

Debt

FL and NFL both have $30,100 of debt on their credit cards and in student loans. We've combined them under the single category called "debt". Each month, the minimum amount that the lending company requires be paid off is 3%. NFL pays off just the minimum 3% each month plus an additional dollar (the additional dollar is important because without it, the function would continue taking 3% of the remaining debt, and consequently would approach 0 but never reach it). FL pays off the minimum 3% plus an additional $15 of it, knowing that paying the debt off more quickly will reduce the amount that they ultimately end up paying. The interest on the debt that hasn't been paid off is 20% compounded annually (this is a simplification of the actual interest rate on credit card debt). This is calculated each year by multiplying the remaining debt by 1.2.

Debt payment should stop on the month that the debt is no longer over $0.

Housing and mortgage loan

Different Down Payments on a House

FL and NFL decide that they'd like to buy houses. The difference is that they decide to buy them at different times in their lives. NFL buys the house as soon as they're able to afford a down payment of 5% on the house (so as soon as their checking is over 5% of the total cost of the house). FL buys the house as soon as they're able to afford a down payment of 20% on the house. Both are buying a house that costs $175,000, and the interest rate for both is 4.5%. What NFL doesn't know is that when you pay a down payment less than 20% on any house, an extra interest rate called "private mortgage insurance" (PMI) is added onto the cost. A .5% (assuming NFL has a great credit score) interest rate is added onto the amount that NFL has to pay each year.

Monthly rent payment

Before each one has enough money in their checking account to pay the 5% or 20% down payment, they're living in apartments that cost them $850 a month (rent comes from checking account).

Monthly mortgage loan payment

Once they have enough for the down payment, "loan" should be updated to be the price of the house minus the down payment, and then subtracting mortgage payment should be run each year until the loan has been paid off. To calculate the monthly mortgage

payment, you can assume that both FL and NFL will pay off their mortgages in full in 30 years and that they pay the same amount of money one year as they'll pay the next year.

Details:

- The interest on the loan for FL is 4.5%. The interest on the loan for NFL is 5% (4.5% base rate plus 0.5% of PMI).
- To calculate the monthly mortgage payment
  - N (# payments) = 360 (30 years times 12 monthly payments per year)
  - i (interest) = (interest divided by 12)
  - D (discount factor) = $((i+1)^N-1) / (i\times(1+i)^N)$
  - P (monthly payment) = Loan amount / D
- The money comes from the checking account. When they don't have enough on the checking account, it should come from the savings account.

(You can read more about buying a house with different down payments here.)

Calculations

You will be calculating the balance of checking and savings account, amount of debt and amount of loan after the end of each year, and total wealth at the end of each passing year. You should track at least: years in debt (including mortgage), years renting, and total amount paid toward debt and, separately, toward mortgage.

Over one year, small choices may not change much; over 40 years, expect the gap to be large.

Simulation conditions (reference)

You may use these numbers as a starting point for your simulation. You should also feel free to adjust them in a consistent way and document your choices. The important part is that **both people start equal** and **differ only in the decisions they make**.

Item	Typical value (for reference)
Initial savings	$5,000
Initial checking	$0
Initial debt (e.g., credit cards + student loans)	$30,100
Annual salary	$61,000 (e.g., median middle-class)
Rent (before buying a house)	$850/month
House price	$175,000
Mortgage	30-year; base rate ~4.5%; extra cost (e.g., PMI) if down payment < 20% is 0.5%
Savings return	e.g., 1% (bank) vs. ~7% (e.g., mutual fund)
Debt interest	e.g., ~20% annually; minimum payment e.g. 3% of balance per month

Required classes and functions (fixed interface)

Your code must be in a file called `main.py`. It must define the following names so that we can grade your work:

`Person` class

`Simulation` class

`plot_wealth` function

`run_tests` function

You are free to add **any additional classes, helper functions, and methods** you like.

`Person` class (flexible design, fixed interface)

You choose:

which attributes to store (savings_balance, checking_balance, debt, has_house, etc.),

---

which methods to have and what they do,

---

how to model the difference between FL and NFL.

Required:

- Constructor:

```
class Person:
    def __init__(self, name: str, is_literate: bool):
        ...
```

  - `name`: a short identifier (e.g., `"fl"` or `"nfl"`).
  - `is_literate`: `True` for financially literate, `False` otherwise.
  - You may add **optional** parameters with default values.
- A way to obtain the person's current **wealth** as a single number:
  - Either a property:

```
@property
def wealth(self) -> int:
    ...
```

  - or a method:

```
def get_wealth(self) -> int:
    ...
```

For grading, we will first try `person.wealth`; if that does not exist we will try `person.get_wealth()`.

Wealth should represent the person's overall financial position, typically something like

`wealth = savings + checking - debt - loan`

rounded to an integer.

You should also keep track of other metrics you consider interesting (e.g., years in debt, total debt paid, years renting, etc.).

`Simulation` class

You construct a `Simulation` by passing in a single **Person** and the number of years to simulate.

Required:

- **Constructor** takes one `Person` and simulation length as parameters:

```
class Simulation:
    def __init__(self, person: Person, years: int = 40):
        ...
```

  - `person` (required): the single `Person` object to simulate.
  - `years` (optional): number of years to simulate; default 40.
  - You may add further optional parameters with defaults.

- **Method** `run`:

```
def run(self) -> list:
    ...
```

  This must:

  - Simulate the given number of years for **this** person, updating the person's state each year.
  - Return a **list** of length `years + 1`: `[w0, w1, ..., wN]`, where `w0` is initial wealth (before any years), `w1` is wealth after 1 year, …, `wN` is wealth after `years` years.

- **Method** `summary`:

```
def summary(self) -> dict:
    ...
```

Return a dictionary for **this** person with at least:

- `"final_wealth"` (int)
- `"years_in_debt"` (refers only to the time person carried credit card and student loan debt)
- `"total_debt_paid"` (total payments toward credit card / student loan debt only)
- `"total_mortgage_paid"` (total payments toward the mortgage loan)

You may include additional keys. (The main program will create two `Simulation` instances (one for FL, one for NFL) and combine their summaries to print the graded `RESULT` line.)

Plotting function

You will create a plot with `matplotlib` showing how wealth changes over time.

Required:

```
def plot_wealth(fl_wealth_history: List, nfl_wealth_history: List, filename: str =
        "wealth_over_time.png") -> None:
    ...
```

- `fl_wealth_history` and `nfl_wealth_history` are two lists of wealth over time for FL and NFL respectively (each of the form `[w0, w1, ...]`; the main program passes the two lists returned by `Simulation.run()`).
- You must:
  - Plot FL and NFL wealth vs. year on the **same figure**.
  - Label axes and include a legend.
  - Save a PNG file with the given `filename` in the current directory (by default `wealth_over_time.png`).

We will check that the file exists and is non-empty, and may view it when grading.

Testing function

You should write your own tests to check your code.

Required:

```
def run_tests() -> None:
    ...
```

When called, it must:

- Create at least one `Person` and one `Simulation`.
- Call key methods and verify behavior using `assert` statements with helpful messages.
- Include at least:
  - 3 asserts related to each method of `Person`.
  - 3 asserts related to `Simulation`.

We will call `run_tests()` and expect it to finish without raising an AssertionError.

Main program behavior

At the bottom of `main.py`, include:

```
if __name__ == "__main__":
    ...
```

Required behavior when we run `python main.py`:

1. Create two persons:

   ```
   fl = Person("fl", True)
   nfl = Person("nfl", False)
   ```

2. Create **two** simulations (one per person) and run each:

   ```
   sim_fl = Simulation(fl, years=40)
   sim_nfl = Simulation(nfl, years=40)
   fl_wealth_history =   sim_fl.run()
   nfl_wealth_history =  sim_nfl.run()
   ```

3. Generate and save the plot:

   ```
   plot_wealth(fl_wealth_history, nfl_wealth_history)   # saves "wealth_over_time.png"
   ```

4. Get summaries from both simulations and print **one machine-readable line** beginning with `RESULT`:

```python
summary_fl = sim_fl.summary()
summary_nfl = sim_nfl.summary()
print(
    f"RESULT "
    f"fl_final_wealth={summary_fl['final_wealth']} "
    f"nfl_final_wealth={summary_nfl['final_wealth']} "
    f"fl_years_in_debt={summary_fl['years_in_debt']} "
    f"nfl_years_in_debt={summary_nfl['years_in_debt']} "
    f"fl_total_debt_paid={summary_fl['total_debt_paid']} "
    f"nfl_total_debt_paid={summary_nfl['total_debt_paid']} "
    f"fl_total_mortgage_paid={summary_fl['total_mortgage_paid']} "
    f"nfl_total_mortgage_paid={summary_nfl['total_mortgage_paid']}"
)
```

Do **not** change the word `RESULT` or the field names; this is for automated grading.

Design freedom ("Your touch")

The above description outlines the standard expectations from the project. Another part of your work is to make your project unique, and uniquely good. It is up to you to decide what you will do. The main requirement is that you go beyond the described requirements in a meaningful way. You could add timing to your processing and display how much time the program takes, use recursive function implementation, raise and process exceptions, read input parameters from file, use another library in some meanignful way, use more classes with inheritance, add more parameters to the simulation representing addition life events (medical emergencies, job loss, promotion with raise, paying for children's education, etc.)

Document your design decisions in comments and in your reflection.

Because of this design freedom opportunity as well as different design decisions (the order in which monthly payments are made might affect the account balances differently), autograding the project is not possible and autograder will not be provided. There is a checker set up on gradescope that verifies that the submitted code aligns with the project description. Passing the checker may not be an indicator of code correctness.

Deliverables

Upload to Gradescope:

`main.py`
(containing `Person`, `Simulation`, `plot_wealth`, `run_tests`, and the `if __name__ == "__main__":` block).

A `readme.pdf` file containing the following clearly labeled elements:

- A class diagram showing attributes of `Person` and `Simulation` classes, methods of both classes, their parameters, parameter types and return types. Attributes, methods with parameter listing, parameter types and return types for any other classes you add.
- The plot with wealth produced by the plotting function
- Answers to the following questions
  - What did you learn from this assignment that you didn't know before?
  - How much more debt did NFL end up paying compared to FL?
  - How many more years was NFL in debt compared to FL?
  - How much more money does FL have at the end of 40 years compared to NFL?
  - Which decision in your model had the greatest effect on the disparity between NFL and FL?
- A brief description of the work done for "your touch" part.
- Any additional notes or instructions and a description of known limitations (optional).

All submitted code must run without errors, non-running code won't receive points. Please remove all commented-out code, use meaningful variable names, each function/method should have a usual docstring, etc. Use constants where applicable, for instance, all numbers that can change (interest rates, loan amounts, salary, rent, house cost etc.) should be constants. Points may be deducted for issues with code quality.

Point distribution

Component	Points
Person class	30
Simulation class	30
Plotting	10
Testing (`run_tests`)	15
Written part with all required components (`readme.pdf`)	10
"Your touch"	5
Total	**100**

Acknowledgements

This assignment is a modified "Financial Literacy" project that was created by Kaitlyn Zeichick and Colleen Lewis and is licensed under CC BY. Accessed from www.engage-csedu.org.